

## Chapter 13

# 조인 튜닝 - 2

# Outline

---

- 13.1 서브쿼리 조인
- 13.2 고급 조인 기법

## 13.1 서브쿼리 조인

### 서브쿼리 변환

옵티마이저는 SQL을 최적화를 하기위해 쿼리 변환(Query Transformation)을 진행한다.

```
SELECT ENAME
, (SELECT X.DNAME
    FROM DEPT X
    WHERE X.DEPTNO = A.DEPTNO) AS DNAME
FROM (SELECT EMPNO, DEPTNO, ENAME
      FROM EMP
      WHERE SAL > 3000) A
WHERE EXISTS (SELECT 1
              FROM DEPT X
              WHERE X.DEPTNO = A.DEPTNO)
```

스칼라 서브쿼리

인라인 뷰

중첩 서브쿼리

- 쿼리변환 : 옵티마이저가 SQL을 분석해 좋은 성능이 나도록 SQL로 변환해 재작성하는 기능

## 13.1 서브쿼리 조인

### 쿼리 최적화

옵티마이저는 SQL을 블록 단위로 최적화를 수행한다.

```
SELECT EMPNO  
      , ENAME  
FROM (SELECT EMPNO, DEPTNO, ENAME  
      FROM EMP  
      WHERE SAL > 3000) A  
WHERE EXISTS (SELECT 1  
             FROM DEPT X  
             WHERE X.DEPTNO = A.DEPTNO)
```

쿼리 블록1

쿼리 블록2

쿼리 블록3

- 위 SQL을 쿼리 변환 하지 않고 수행하면 쿼리 블록 단위로 각각 최적화 한다.

## 13.1 서버쿼리 조인

### 쿼리 최적화

### 쿼리 최적화 대상 블록들

#### 쿼리 블록1

```
SELECT EMPNO  
      , ENAME  
FROM SYS_VW_TEMP A
```

#### 쿼리 블록2

```
SELECT EMPNO, DEPTNO, ENAME  
FROM EMP  
WHERE SAL > 3000
```

#### 쿼리 블록3

```
SELECT 1  
FROM DEPT X  
WHERE X.DEPTNO = A.DEPTNO
```

각각 최적화 되었다고 해도 전체 쿼리가 최적화 되었다고 보장 할 수 없음

## 13.1 서브쿼리 조인

### 서브쿼리와 조인

메인쿼리와 서브쿼리 간에는 부모, 자식으로 종속적이므로 메인쿼리의 건수만큼 반복적으로 필터링 하는 방식으로 실행이 필요하다.

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT EMPNO, ENAME
FROM EMP A
WHERE A. SAL > 3000
      AND EXISTS (SELECT /*+ no_unnest */1
                  FROM DEPT X
                  WHERE X.DEPTNO = A.DEPTNO);
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 1    | 17    | 2 (0)       | 00:00:01 |
| * 1 | <b>FILTER</b>                       |            |      |       |             |          |
| 2   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 1    | 17    | 2 (0)       | 00:00:01 |
| * 3 | INDEX SKIP SCAN                     | IDX_EMP_06 | 1    |       | 1 (0)       | 00:00:01 |
| * 4 | INDEX UNIQUE SCAN                   | PK_DEPT    | 1    | 3     | 0 (0)       | 00:00:01 |

- **필터오퍼레이션** : 서브쿼리를 필터방식으로 처리할 때 실행 계획
- **no\_unnest** : 서브쿼리를 현재 상태에서 최적화(메인쿼리 병합 X)

## 13.1 서브쿼리 조인

### 필터 오퍼레이션 특징

| 특징       | 설명                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 부분 범위 처리 | <ul style="list-style-type: none"><li>- 메인 쿼리의 한 로우가 서브쿼리의 한 로우와 조인이 되는 순간 진행을 멈추고 메인 쿼리는 다음 로우를 처리함</li></ul> <pre>BEGIN   FOR OUTER IN (SELECT EMPNO, ENAME FROM EMP A WHERE SAL &gt; 3000)   LOOP     FOR INNER IN (SELECT 1 FROM DEPT X WHERE X.DEPTNO = OUTER.DEPTNO)     LOOP       dbms_output.put_line(OUTER.EMPNO    ','    OUTER.ENAME);       EXIT; /* 조인 성공하면 inner loop 종료 */     END LOOP   END LOOP END</pre> <ul style="list-style-type: none"><li>- 위와 같이 Inner loop에서 조인을 성공하면 exit 로 종료한다.</li></ul> |
| 캐싱       | <ul style="list-style-type: none"><li>- 서브쿼리 반환 값 또는 결과를 캐싱</li><li>- 이 때문에 서브쿼리 실행 전 캐시부터 확인</li><li>- 캐시에서 결과를 찾으면 서브쿼리를 수행하지 않아도 되기 때문에 성능이 향상됨</li></ul>                                                                                                                                                                                                                                                                                                                                                            |
| 조인 순서 고정 | <ul style="list-style-type: none"><li>- 메인 쿼리에 종속되므로 조인순서가 고정됨</li><li>- 메인 쿼리가 항상 드라이빙 테이블이 됨</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                |

## 13.1 서브쿼리 조인

### Unnesting

메인쿼리와 서브쿼리 간의 계층구조를 풀어내 같은 레벨로 만들어 주는 힌트로, 서브쿼리 Flattening이라고도 한다.

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT EMPNO, ENAME
FROM EMP A
WHERE A. SAL > 3000
      AND EXISTS (SELECT /*+ unnest nl_sj */1
                  FROM DEPT X
                  WHERE X.DEPTNO = A.DEPTNO AND X.DNAME LIKE 'A%');
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 1    | 30    | 3 (0)       | 00:00:01 |
| 1   | NESTED LOOPS SEMI                   |            | 1    | 30    | 3 (0)       | 00:00:01 |
| 2   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 1    | 17    | 2 (0)       | 00:00:01 |
| * 3 | INDEX SKIP SCAN                     | IDX_EMP_06 | 1    |       | 1 (0)       | 00:00:01 |
| * 4 | TABLE ACCESS BY INDEX ROWID         | DEPT       | 1    | 13    | 1 (0)       | 00:00:01 |
| * 5 | INDEX UNIQUE SCAN                   | PK_DEPT    | 1    |       | 0 (0)       | 00:00:01 |

- **unnest** : 서브쿼리를 메인쿼리와 병합하여 최적화
- **nl\_sj** : NL 세미조인 방식으로 실행

## 13.1 서브쿼리 조인

### Unnesting

Unnesting 서브쿼리를 드라이빙 테이블로 사용할 경우 실행계획

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT /*+ leading(X@subj) use_nl(A) */
      EMPNO, ENAME
FROM EMP A
WHERE A. SAL > 3000
      AND EXISTS (SELECT /*+ qb_name(subj) nl_sj */1
                  FROM DEPT X
                   WHERE X.DEPTNO = A.DEPTNO AND X.DNAME LIKE 'A%');
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 1    | 30    | 3 (0)       | 00:00:01 |
| 1   | NESTED LOOPS SEMI                   |            | 1    | 30    | 3 (0)       | 00:00:01 |
| 2   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 1    | 17    | 2 (0)       | 00:00:01 |
| * 3 | INDEX SKIP SCAN                     | IDX_EMP_06 | 1    |       | 1 (0)       | 00:00:01 |
| * 4 | TABLE ACCESS BY INDEX ROWID         | DEPT       | 1    | 13    | 1 (0)       | 00:00:01 |
| * 5 | INDEX UNIQUE SCAN                   | PK_DEPT    | 1    |       | 0 (0)       | 00:00:01 |

– qb\_name : Query Block에 이름을 지정

## 13.1 서브쿼리 조인

### Unnesting

HASH 세미조인 방식으로 실행해야 할 경우 실행계획

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT EMPNO, ENAME
FROM EMP A
WHERE A. SAL > 3000
      AND EXISTS (SELECT /*+ unnest hash_sj */1
                  FROM DEPT X
                  WHERE X.DEPTNO = A.DEPTNO AND X.DNAME LIKE 'A%');
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 1    | 30    | 5 (0)       | 00:00:01 |
| * 1 | HASH JOIN SEMI                      |            | 1    | 30    | 5 (0)       | 00:00:01 |
| 2   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 1    | 17    | 2 (0)       | 00:00:01 |
| * 3 | INDEX SKIP SCAN                     | IDX_EMP_06 | 1    |       | 1 (0)       | 00:00:01 |
| * 4 | TABLE ACCESS FULL                   | DEPT       | 1    | 13    | 3 (0)       | 00:00:01 |

– `hash_sj` : HASH 세미조인 방식으로 실행

## 13.1 서브쿼리 조인

### Pushing

Unnesting 되지 않은 서브쿼리는 필터방식으로 처리되기 때문에 힌트를 사용해 앞단계에서 처리되도록 강제하는 방식

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT A.EMPNO, A.ENAME, B.EMPNO, B.ENAME
FROM EMP A, EMP B
WHERE A.SAL > 3000 AND B.EMPNO = A.MGR
AND EXISTS (SELECT /*+ NO_UNNEST */1
            FROM DEPT X
            WHERE X.DEPTNO = A.DEPTNO AND X.DNAME LIKE 'A%');
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 1    | 31    | 4 (0)       | 00:00:01 |
| * 1 | <b>FILTER</b>                       |            |      |       |             |          |
| 2   | NESTED LOOPS                        |            |      |       |             |          |
| 3   | NESTED LOOPS                        |            |      |       |             |          |
| * 4 | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        |      |       |             |          |
| * 5 | INDEX SKIP SCAN                     | IDX_EMP_06 |      |       |             |          |
| * 6 | INDEX UNIQUE SCAN                   | PK_EMP     |      |       |             |          |
| 7   | TABLE ACCESS BY INDEX ROWID         | EMP        |      |       |             |          |
| * 8 | TABLE ACCESS BY INDEX ROWID         | DEPT       |      |       |             |          |
| * 9 | INDEX UNIQUE SCAN                   | PK_DEPT    | 1    |       | 0 (0)       | 00:00:01 |

메인집합이 먼저 조인이 되고  
필터링 처리됨

-> 먼저 조인이 되는 만큼  
읽어야 하는 블록 수 증가

## 13.1 서브쿼리 조인

### Pushing

Unnesting 되지 않은 서브쿼리는 필터방식으로 처리되기 때문에 힌트를 사용해 앞단계에서 처리되도록 강제하는 방식

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT A.EMPNO, A.ENAME, B.EMPNO, B.ENAME
FROM EMP A, EMP B
WHERE A.SAL > 3000 AND B.EMPNO = A.MGR
AND EXISTS (SELECT /*+ NO_UNNEST PUSH_SUBQ */1
            FROM DEPT X
            WHERE X.DEPTNO = A.DEPTNO AND X.DNAME LIKE 'A%');
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 1    | 31    | 4 (0)       | 00:00:01 |
| 1   | NESTED LOOPS                        |            | 1    | 31    | 3 (0)       | 00:00:01 |
| 2   | NESTED LOOPS                        |            |      |       |             |          |
| * 3 | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        |      |       |             |          |
| * 4 | INDEX SKIP SCAN                     | IDX_EMP_06 |      |       |             |          |
| * 5 | TABLE ACCESS BY INDEX ROWID         | DEPT       |      |       |             |          |
| * 6 | INDEX UNIQUE SCAN                   | PK_DEPT    |      |       |             |          |
| * 7 | INDEX UNIQUE SCAN                   | PK_EMP     |      |       |             |          |
| 8   | TABLE ACCESS BY INDEX ROWID         | EMP        |      |       |             |          |

메인집합이 먼저 필터링 되고  
조인이 실행  
-> 필터링 후 조인되기 때문에  
조인횟수, 읽은 블록 수 감소

## 13.1 서브쿼리 조인

### VIEW 조인

옵티마이저가 뷰 쿼리를 변환하지 않으면 뷰 쿼리 블록을 독립적으로 최적화를 진행한다.

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT A.EMPNO, A.ENAME, B.SAL
FROM EMP A
      , (SELECT /*+ no_merge */X.DEPTNO, SUM(X.SAL) AS SAL
          FROM EMP X, DEPT Y WHERE Y.DEPTNO = X.DEPTNO
          GROUP BY X.DEPTNO) B
WHERE A.SAL > 1000 AND B.DEPTNO = A.DEPTNO;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 11   | 473   | 4 (0)       | 00:00:01 |
| * 1 | HASH JOIN                           |            | 11   | 473   | 4 (0)       | 00:00:01 |
| 2   | VIEW                                |            | 3    | 78    | 2 (0)       | 00:00:01 |
| 3   | HASH GROUP BY                       |            |      |       |             |          |
| 4   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        |      |       |             |          |
| * 5 | INDEX FULL SCAN                     | IDX_EMP_05 |      |       |             |          |
| 6   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        |      |       |             |          |
| * 7 | INDEX SKIP SCAN                     | IDX_EMP_06 | 10   |       | 1 (0)       | 00:00:01 |

뷰 최적화를 수행 후 EMP A  
테이블과 조인을 처리

## 13.1 서브쿼리 조인

### VIEW 조인

옵티마이저가 뷰 쿼리를 변환하게 되면 NL, HASH 조인으로 변환해서 실행 계획을 수립한다.

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT A.EMPNO, A.ENAME, B.SAL
FROM EMP A
      , (SELECT /*+ merge */X.DEPTNO, SUM(X.SAL) AS SAL
          FROM EMP X, DEPT Y WHERE Y.DEPTNO = X.DEPTNO
          GROUP BY X.DEPTNO) B
WHERE A.SAL > 1000 AND B.DEPTNO = A.DEPTNO;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 45   | 1620  | 5 (20)      | 00:00:01 |
| 1   | HASH GROUP BY                       |            | 45   | 1620  | 5 (20)      | 00:00:01 |
| * 2 | HASH JOIN                           |            | 45   | 1620  | 4 (0)       | 00:00:01 |
| 3   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        |      |       |             |          |
| * 4 | INDEX SKIP SCAN                     | IDX_EMP_06 |      |       |             |          |
| 5   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        |      |       |             |          |
| * 6 | INDEX FULL SCAN                     | IDX_EMP_01 |      |       |             |          |

뷰 머징을 통해 쿼리 변환 후  
HASH JOIN으로 실행계획 수립

## 13.1 서브쿼리 조인

### 조인조건 Pushdown

메인쿼리의 조인조건을 VIEW 안에 조인조건으로 추가하는 기능

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT A.EMPNO, A.ENAME, B.SAL
FROM EMP A
      , (SELECT /*+ no_merge push_pred */X.DEPTNO, SUM(X.SAL) AS SAL
          FROM EMP X, DEPT Y WHERE Y.DEPTNO = X.DEPTNO
          GROUP BY X.DEPTNO) B
WHERE A.SAL > 1000 AND B.DEPTNO = A.DEPTNO;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 11   | 330   | 22 (0)      | 00:00:01 |
| 1   | NESTED LOOPS                        |            | 11   | 330   | 22 (0)      | 00:00:01 |
| 2   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 10   | 170   | 2 (0)       | 00:00:01 |
| * 3 | INDEX SKIP SCAN                     | IDX_EMP_06 | 10   |       | 1 (0)       | 00:00:01 |
| 4   | VIEW PUSHED PREDICATE               |            |      |       |             | 00:00:01 |
| * 5 | FILTER                              |            |      |       |             |          |
| 6   | SORT AGGREGATE                      |            |      |       |             |          |
| 7   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 4    | 20    | 4 (0)       | 00:00:01 |
| * 8 | INDEX RANGE SCAN                    | IDX_EMP_05 | 4    |       | 1 (0)       | 00:00:01 |

VIEW PUSHED PREDICATE

오퍼레이션을 통해 조인조건 PUSHDOWN 확인

- push\_pred : 조인 조건 컬럼을 뷰 안으로 병합하는 힌트

## 13.1 서브쿼리 조인

### 스칼라 서브쿼리 조인

스칼라 서브쿼리를 테스트 하기 위해 function을 생성한다.

```
CREATE OR REPLACE FUNCTION FN_GET_DNAME (p_deptno NUMBER)
RETURN VARCHAR2
IS
    v_dname VARCHAR(14);
BEGIN
    SELECT dname INTO v_dname FROM dept WHERE deptno = p_deptno;
    RETURN v_dname;
END;
```

```
SELECT EMPNO, ENAME, FN_GET_DNAME(DEPTNO) AS DNAME
FROM EMP WHERE SAL > 3000;
```

- 위 SQL을 실행하면 SELECT 절의 함수 내부의 SELECT 조회 쿼리가 메인 쿼리의 개수만큼 반복 실행 된다.

## 13.1 서브쿼리 조인

### 스칼라 서브쿼리 조인 캐싱

스칼라 서브쿼리는 메인쿼리 레코드마다 하나의 값을 반환하지만 함수처럼 메인 쿼리의 개수만큼 반복실행 되지 않음

```
SELECT EMPNO, ENAME,  
       (SELECT dname FROM dept WHERE deptno = A.deptno) AS DNAME  
FROM EMP A WHERE SAL > 3000;
```

- 입력 값과 출력 값을 내부 캐시(Query Execution Cache)에 저장 후 조인할 때마다 캐시에서 입력 값을 찾아보고 출력 값을 반환한다.
- 이로 인해 조인 수행 횟수를 최소화 할 수 있기 때문에 조인 성능 향상된다.

```
SELECT EMPNO, ENAME,  
       (SELECT FN_GET_DNAME(DEPTNO) FROM dual) AS DNAME  
FROM EMP A WHERE SAL > 3000;
```

- FUNCTION에 스칼라 서브쿼리로 감싸면 호출 횟수를 최소화 할 수 있다.
- 스칼라 서브쿼리 캐싱 효과이기 때문이다.

## 13.1 서브쿼리 조인

### 스칼라 서브쿼리 조인 캐싱 부작용

캐시 공간은 한정적이기 때문에 스칼라 서브쿼리의 입력값 종류가 적어야지 효율이 높다.

```
SELECT A.주문번호, A.고객번호,  
       (SELECT X.코드명 FROM 통합코드 X WHERE X.코드 = A.주문구분코드)  
FROM 주문 A  
WHERE A.주문일자 BETWEEN SYSDATE - 7 AND SYSDATE;
```

- 위 SQL에서는 주문구분코드 개수만큼만 조인을 시도하고 그 이후에는 캐시에서 데이터를 찾게 되므로 조인 성능이 높아진다.

```
SELECT A.주문번호, A.고객번호,  
       (SELECT X.카드번호 FROM 결제수단 X WHERE X.결제수단ID = A.결제수단ID)  
FROM 주문 A  
WHERE A.주문일자 BETWEEN SYSDATE - 7 AND SYSDATE;
```

주문건수만큼 실행  
→ 불필요한 캐시 탐색

- 위 SQL 처럼 주문 건별로 결제수단으로 조회 할 경우 매번 캐시탐색을 하지만 데이터를 찾지 못해 조인을 하게 됨으로, 불필요한 캐시탐색을 한다.
- 이럴 경우 NL 조인으로 변경을 하는 것이 성능상 유리하다.

## 13.1 서브쿼리 조인

### 스칼라 서브쿼리 조인 캐싱 부작용

메인쿼리 집합 결과가 작을 경우 비효율적이다. 서브쿼리 캐싱은 쿼리 단위로 이뤄지는데 메인쿼리 집합 결과가 작을 경우 캐시 재사용성이 낮아진다.

```
SELECT A.주문번호, A.고객번호,  
       (SELECT X.코드명 FROM 통합코드 X WHERE X.코드 = A.주문구분코드)  
FROM 주문 A  
WHERE A.주문일자 BETWEEN SYSDATE - 7 AND SYSDATE  
AND A.고객ID = :고객ID;
```

- 위 SQL은 고객의 최근 일주일간 주문한 결과를 보여주는 쿼리
- 고객이 일주일간 주문한 주문건수가 일반적으로 많지 않기 때문에 **쓰지도 않을 캐시를 할당 후 버리는 결과가 발생**할 수 있다.
- 쿼리의 실행빈도가 적을 경우에는 미비하지만 실행빈도가 높은 쿼리인 경우에는 시스템에 영향을 미칠 수 있다.

## 13.1 서브쿼리 조인

### 스칼라 서브쿼리 조인 두개 이상 값 반환

스칼라 서브쿼리는 기본적으로 NL조인과 유사한 실행계획 수립

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT EMPNO, ENAME,
(SELECT AVG(X.SAL) AS SAL FROM EMP X WHERE X.DEPTNO=A.DEPTNO) AS SAL
FROM EMP A WHERE SAL > 3000;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 2    | 72    | 4 (25)      | 00:00:01 |
| 1   | HASH GROUP BY                       |            | 2    | 72    | 4 (25)      | 00:00:01 |
| 2   | NESTED LOOPS OUTER                  |            | 4    | 144   | 3 (0)       | 00:00:01 |
| 3   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 1    | 29    | 2 (0)       | 00:00:01 |
| * 4 | INDEX SKIP SCAN                     | IDX_EMP_06 | 1    |       | 1 (0)       | 00:00:01 |
| 5   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 5    | 35    | 1 (0)       | 00:00:01 |
| * 6 | INDEX RANGE SCAN                    | IDX_EMP_05 | 4    |       | 0 (0)       | 00:00:01 |

- NL 조인과 다른점은 캐싱 효과를 가진다는 점이다.
- 하지만 NL조인과 다른 점은 두개 이상 값 반환이 어렵다는 점이다.
- 부분범위처리의 장점을 가지는 스칼라 서브쿼리의 장점을 활용을 못하기 때문이다.

## 13.1 서브쿼리 조인

### 스칼라 서브쿼리 조인 두개 이상 값 반환

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT EMPNO, ENAME,
(SELECT AVG(SAL) AS SAL FROM EMP X WHERE X.DEPTNO=A.DEPTNO) AS SAL1,
(SELECT MAX(SAL) AS SAL FROM EMP X WHERE X.DEPTNO=A.DEPTNO) AS SAL2
FROM EMP A WHERE SAL > 3000;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

– 위와 같이 작성하면 EMP 테이블의 같은 데이터를 반복해서 읽는 비효율발생

```
SELECT A.EMPNO, A.ENAME
, SUBSTR(SAL1, 1, INSTR(SAL1, '|')-1)
, SUBSTR(SAL1, INSTR(SAL1, '|')+1)
FROM (SELECT EMPNO, ENAME,
(SELECT AVG(X.SAL) || '|' || MAX(X.SAL) AS SAL
FROM EMP X WHERE X.DEPTNO=A.DEPTNO) AS SAL1
FROM EMP A WHERE SAL > 3000
) A
```

– 위와 같이 문자열을 결합하고 바깥쪽 SELECT 절에서 SUBSTR 함수를 사용해 분리하는 방식으로 두개 이상 값 반환이 가능하다.

## 13.1 서브쿼리 조인

### 스칼라 서브쿼리 조인 두개 이상 값 반환

인라인 뷰를 사용하고자 할 경우에는 조인조건 Pushdown 사용

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT A.EMPNO, A.ENAME, B.SAL1, B.SAL2
FROM EMP A
      , (SELECT /*+ no_merge push_pred */ X.DEPTNO, AVG(X.SAL) AS SAL1
          , MAX(X.SAL) AS SAL2
          FROM EMP X GROUP BY X.DEPTNO) B
WHERE A.SAL > 1000
AND A.DEPTNO = B.DEPTNO;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 11   | 473   | 22 (0)      | 00:00:01 |
| 1   | NESTED LOOPS                        |            | 11   | 473   | 22 (0)      | 00:00:01 |
| 2   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 10   | 170   | 2 (0)       | 00:00:01 |
| * 3 | INDEX SKIP SCAN                     | IDX_EMP_06 | 10   |       | 1 (0)       | 00:00:01 |
| 4   | VIEW PUSHED PREDICATE               |            | 1    | 26    | 2 (0)       | 00:00:01 |
| * 5 | FILTER                              |            |      |       |             |          |
| 6   | SORT AGGREGATE                      |            | 1    | 7     |             |          |
| 7   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 4    | 28    | 2 (0)       | 00:00:01 |
| * 8 | INDEX RANGE SCAN                    | IDX_EMP_05 | 4    |       | 1 (0)       | 00:00:01 |

## 13.1 서브쿼리 조인

### 스칼라 서브쿼리 Unnesting

스칼라 서브쿼리도 NL방식으로 조인하기 때문에 캐시효과가 크지 않으면 랜덤 I/O 부담이 있다. (특히, 병렬 쿼리에서 주의)  
그래서 unnesting을 활용하면 HASH JOIN으로 실행계획 수립

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT EMPNO, ENAME,
(SELECT /*+ unnest no_merge */ AVG(X.SAL) AS SAL
FROM EMP X WHERE X.DEPTNO=A.DEPTNO) AS SAL
FROM EMP A WHERE SAL > 3000;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

| Id  | Operation                           | Name       | Rows | Bytes | Cost (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|-------------|----------|
| 0   | SELECT STATEMENT                    |            | 2    | 72    | 4 (25)      | 00:00:01 |
| 1   | <b>HASH GROUP BY</b>                |            | 2    | 72    | 4 (25)      | 00:00:01 |
| 2   | NESTED LOOPS OUTER                  |            | 4    | 144   | 3 (0)       | 00:00:01 |
| 3   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 1    | 29    | 2 (0)       | 00:00:01 |
| * 4 | INDEX SKIP SCAN                     | IDX_EMP_06 | 1    |       | 1 (0)       | 00:00:01 |
| 5   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 5    | 35    | 1 (0)       | 00:00:01 |
| * 6 | INDEX RANGE SCAN                    | IDX_EMP_05 | 4    |       | 0 (0)       | 00:00:01 |

스칼라 서브쿼리지만 Unnesting 되었기 때문에 HASH JOIN 됨

## 13.1 서브쿼리 조인

### 스칼라 서브쿼리 Unnesting

Unnesting을 원하지 않을 경우 no\_unnest 힌트를 사용

```
EXPLAIN PLAN SET STATEMENT_ID = 'PLAN1' INTO PLAN_TABLE FOR
SELECT EMPNO, ENAME,
(SELECT /*+ no_unnest */ AVG(X.SAL) AS SAL
 FROM EMP X WHERE X.DEPTNO=A.DEPTNO) AS SAL
FROM EMP A WHERE SAL > 3000;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(null,null,'ALL'));
```

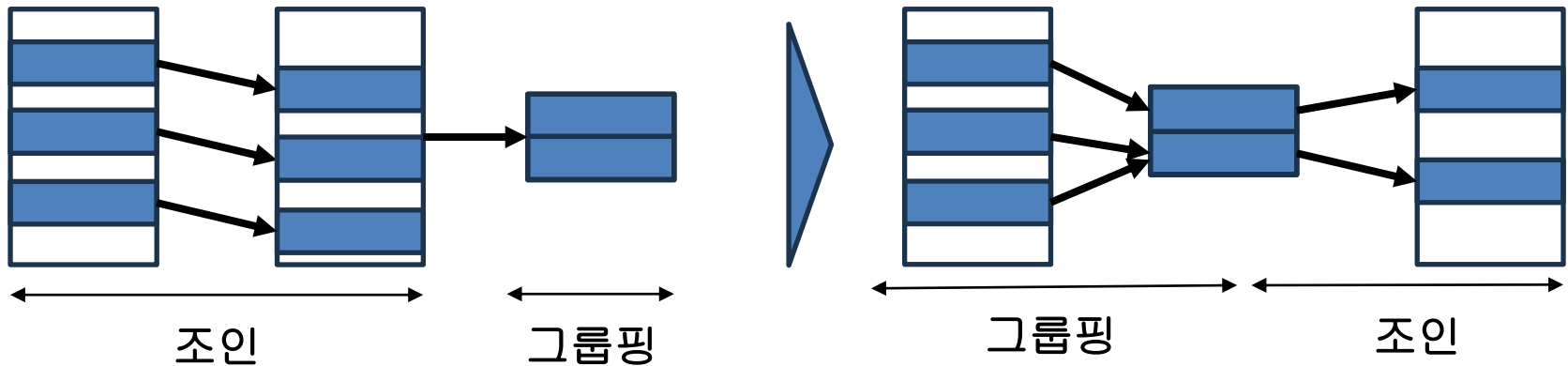
| Id  | Operation                           | Name       | Rows | Bytes | Cost | (%CPU) | Time     |
|-----|-------------------------------------|------------|------|-------|------|--------|----------|
| 0   | SELECT STATEMENT                    |            | 1    | 17    | 4    | (0)    | 00:00:01 |
| 1   | <b>SORT AGGREGATE</b>               |            | 1    | 7     |      |        |          |
| 2   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 4    | 28    | 2    | (0)    | 00:00:01 |
| * 3 | INDEX RANGE SCAN                    | IDX_EMP_05 | 4    |       | 1    | (0)    | 00:00:01 |
| 4   | TABLE ACCESS BY INDEX ROWID BATCHED | EMP        | 1    | 17    | 2    | (0)    | 00:00:01 |
| * 5 | INDEX SKIP SCAN                     | IDX_EMP_06 | 1    |       | 1    | (0)    | 00:00:01 |

Unnesting이 유리하지 않을 경우 no\_unnest 힌트를 사용

## 13.2 고급 조인 기법

### 인라인 뷰 활용

일반적으로 조인을 하고 그룹핑을 할 경우 조인에 참여하는 두 테이블 간에 조인을 시도 한 뒤에 그룹핑을 하게 된다.

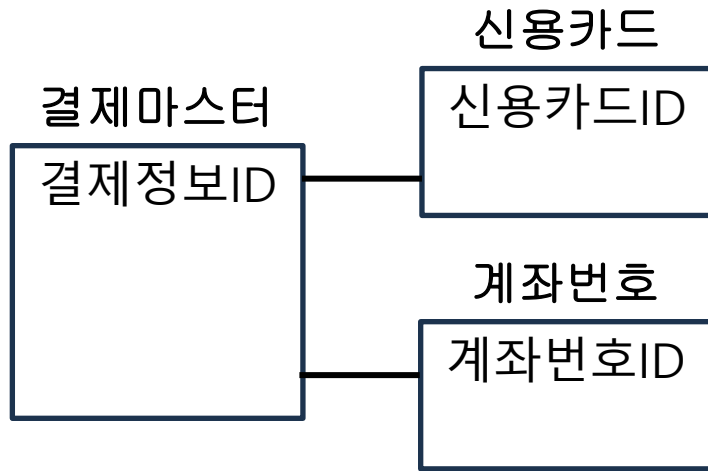


- 조인 후 그룹핑을 하게 될 경우 1:N 관계의 경우 **N의 개수 만큼 조인을 한 뒤, 그룹핑**을 하게 된다.
- 이 때 N의 개수 만큼 조인할때 테이블 Access 가 많이 발생됨
- 그룹핑 후 조인을 하게 되면 **Access 비용 감소**

## 13.2 고급 조인 기법

### 배타적 관계의 조인

어떤 엔터티가 두 개 이상의 다른 엔터티와 관계를 가지지만 한 데이터가 동시에 관계를 가지지 못하는 관계 (상호배타적 관계)



결제마스터는 결제정보가 신용카드 또는 계좌번호가 한 건 씩 관계를 맺고 저장된다.

```
SELECT * FROM 결제마스터 A, 신용카드 B WHERE A.구분코드='1' AND A.결제정보ID=B.신용카드ID  
UNION ALL
```

```
SELECT * FROM 결제마스터 A, 계좌정보 B WHERE A.구분코드='2' AND A.결제정보ID=B.계좌정보ID
```

- 구분코드, 결제정보ID로 인덱스가 구성되어 있다고 가정을 하면 UNION ALL 사용 시 스캔범위에 중복이 생기지 않게 조인이 가능하다.

## 13.2 고급 조인 기법

### 누적 매출 구하기

누적 매출을 구하기 테스트 위한 VIEW를 생성

```
CREATE OR REPLACE VIEW VW_SAL AS
SELECT A.DEPTNO, B.DNAME
, ROW_NUMBER() OVER(PARTITION BY A.DEPTNO ORDER BY A.DEPTNO) AS YEAR
, SAL
FROM EMP A , DEPT B
WHERE B.DEPTNO = A.DEPTNO;
```

뷰 생성이 안되면 권한 부여  
grant create view to scott;

### 윈도우 함수 이용

```
SELECT DNAME AS 부서명, YEAR AS 년, SAL AS 매출
, SUM(SAL) OVER (PARTITION BY DEPTNO ORDER BY YEAR RANGE BETWEEN
UNBOUNDED PRECEDING AND CURRENT ROW) AS 누적매출
FROM VW_SAL;
```

- 윈도우 함수의 분석함수를 이용해 매출 구하기

## 13.2 고급 조인 기법

### 누적 매출 구하기

윈도우 함수를 이용하지 않을 경우

```
•SELECT A.DNAME AS 부서명  
      , A.YEAR AS 년  
      , MIN(A.SAL) AS 매출  
      , SUM(B.SAL) AS 누적매출  
FROM VW_SAL A, VW_SAL B  
WHERE B.DNAME = A.DNAME  
AND B.YEAR <= A.YEAR  
GROUP BY A.DNAME, A.YEAR  
ORDER BY A.DNAME, A.YEAR
```

- 셀프조인과 부등호(<, >) 조인을 통해 누적 매출을 구하기